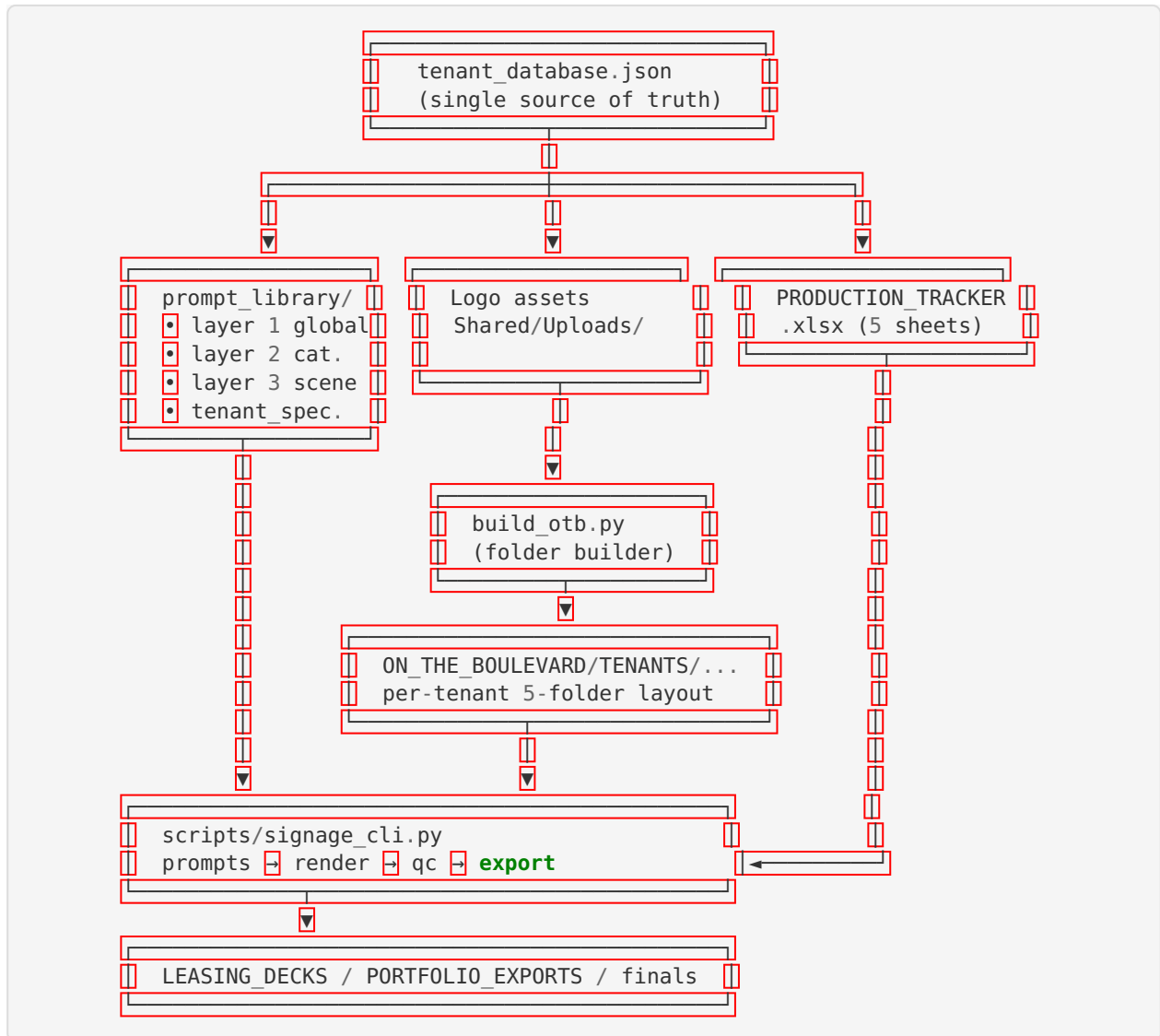


System Architecture — On The Boulevard Rendering Pipeline

1. Overview Diagram



2. Components

2.1 Data Layer

- **tenant_database.json** — canonical tenant list (25 records). Every other component reads from here. Schema fields: `tenant_name`, `unit_number`, `category`, `facade_family`, `logo_file`, `illumination_type`, `notes`.

2.2 Prompt Library (/home/ubuntu/prompt_library/)

- **Layer 1 — Global style:** photoreal architecture, lens, lighting baseline.
- **Layer 2 — Category:** 8 files (boutique, restaurant, fitness, retail, salon, financial, medical, services). Controls signage tone & material.

- **Layer 3 — Scene:** 6 files (daytime, dusk, close_up, streetscape, corner_suite, inline). Controls framing & atmosphere.
- **Tenant-specific overrides:** `tenant_specific/{slug}.txt` for one-off corrections.

2.3 Automation Layer (`/home/ubuntu/scripts/`)

Script	Role
<code>signage_cli.py</code>	Single entry point for all operator commands
<code>generate_prompts.py</code>	Stacks the prompt layers per tenant/scene
<code>generate_renders.py</code>	Calls AI image generator + saves output
<code>qc_check.py</code>	Validates resolution, aspect, integrity
<code>export_package.py</code>	Builds leasing/investor/broker zip packages
<code>production_manager.py</code>	Cross-tenant batch + status
<code>organize_files.py</code>	Moves files between renders/finals/revisions
<code>utils.py</code>	Shared helpers

2.4 Production Layer (`/home/ubuntu/ON_THE_BOULEVARD/`)

- Per-tenant folders (`logo_source` , `prompts` , `renders` , `finals` , `revisions`)
- Cross-cutting folders: `SITE_PLANS` , `LEASING_DECKS` , `PORTFOLIO_EXPORTS` , `BRAND_SYSTEM` , `MASTER_PROMPTS` (symlink)
- Tracker workbook + CSV mirror

3. Data Flow

1. Operator updates `tenant_database.json` .
2. `build_otb.py` provisions folders + copies logos.
3. `signage_cli.py prompts` stacks prompt layers → writes to `prompts/` .
4. `signage_cli.py render` generates images → writes to `renders/` .
5. `signage_cli.py qc` validates → operator logs results in tracker.
6. Operator promotes approved files: `renders/` → `finals/` .
7. `signage_cli.py export` zips deliverables → `LEASING_DECKS/` or `PORTFOLIO_EXPORTS/` .

4. Integration Points

Integration	How
AI image generation	Pluggable provider behind <code>generate_renders.py</code>
Excel tracker	<code>openpyxl</code> write; operators edit manually after generation
PDF/Pptx exports	<code>export_package.py</code> (template-driven)
Site plan overlays	Drop in <code>SITE_PLANS/</code> ; referenced by exports

5. Scalability Considerations

- **Horizontal:** the per-tenant folder model scales linearly. 100 tenants $\approx$ same workflow; only batch render time grows.
- **Prompt reuse:** layered prompts mean adding a new category is one file, not 25.
- **Statelessness:** every script can be re-run idempotently — folders are created only if missing, renders are timestamped (no overwrites).
- **Storage:** each tenant $\approx$ 20–60 MB across 4 scenes $\times$ 2k renders. Plan $\sim$ 2 GB for a fully rendered 25-tenant set.

6. Future Enhancement Roadmap

- **v1.1** — Web UI for the tracker (replaces manual Excel editing).
- **v1.2** — Auto-QC: pixel-level checks for halo-lit vs face-lit illumination.
- **v1.3** — Live site-plan overlays: drop renders onto the plat automatically.
- **v1.4** — Multi-property support (one DB, many properties).
- **v1.5** — Operator notifications (Slack/email) on batch completion.
- **v2.0** — Versioned prompt library with diff/rollback.